Operating Systems — Assignment 9 Report

Manav Singla, 2402MC02 — xv6 (x86, mit-pdos/xv6-public), qemu-system-i386 — Page tables, lazy allocation, page replacement

Q1 — Printing the page table: `vmprint()`

Changes: `walkpgdir()` in `vm.c` was made non-static and declared in `defs.h`. The new `vmprint(pgdir)` follows the structure of `freevm()`: it loops over the 1024 page-directory entries, skips anything at or above `KERNBASE`, and for every present user PDE uses `walkpgdir()` on each of the 1024 VAs it covers. Each valid page prints its VA, frame PA (`PTE_ADDR`) and P/W/U bits. It ends with the number of valid user pages and the page-table pages (1 directory + user page tables), and returns the page count. Plumbing: `sys_vmprint` in `sysproc.c`, plus `syscall.h` / `syscall.c` / `usys.S` / `user.h`.

Output (`pttest`): at start-up there are 3 valid pages (code/data 0x0, guard page 0x1000 with U=0, stack 0x2000) and 2 page-table pages. Right after `sbrk(4*4096)` still 3, because with Q2's lazy sbrk only `sz` grows. Once the 4 pages are first used, 0x3000–0x6000 appear: **7 pages = exactly 4 new**, still in the same page table.

Q2 — Page-fault handling (lazy allocation)

- **sysproc.c:** `sys_sbrk` with `n > 0` only raises `myproc()->sz` (rejecting sizes reaching `KERNBASE`). Shrinking still uses `growproc()` / `deallocuvm()`, which already skips absent pages.
- **vm.c:** `copyuvm()` now `continue`s on a missing page table or a non-present PTE instead of panicking, so fork works with a sparse heap. New `lazyalloc()` does `kalloc()`, `memset(0)`, `mappages(PTE_W|PTE_U)`.
- **trap.c (`T_PGFLT`):** reads `rcr2()`. If `va < sz` and the page is *not present* (`tf->err` bit 0 clear), it allocates, does `p->nfaults++` and resumes. Otherwise it prints the pid and address and sets `killed`. The "not present" test matters: the guard page lies below `sz` but is mapped (U cleared), so remapping it would panic with "remap". A fault there is a protection violation and kills the process.
- **proc.h / proc.c / exec.c:** new field `int nfaults`, zeroed in `allocproc()` and in `exec()`. The exec reset is needed because the shell's child calls `malloc` (lazy sbrk) before exec. `sys_getfaults()` returns it.

Output (`faulttest`): `sbrk(32*4096)` causes 0 faults. Writing pages 0, 3, 10, 20 and 31 raises the count 1→5, and writing the same pages again stays at 5. **Result 5 → PASS.** A child touching 0x24000 (above sz 0x23000) gets "invalid page fault ... killed", while the parent keeps running with its data intact. The stock `usertests` still prints ALL TESTS PASSED.

Q3 — FIFO vs LRU page replacement (`pagerepl.c`)

Design: usage is `pagerepl [frames] [refs...]`. References can be separate arguments or one comma-separated argument, since the xv6 shell allows only 10 arguments. With no input it uses the assignment string. **FIFO** keeps a circular pointer `next`: on a fault, the page at `next` is replaced and the pointer advances. **LRU** stores `last[i]`, the time of the last use of each frame: an empty frame is filled first, otherwise the frame with the smallest `last` is evicted, and every hit or fault updates `last`. Each reference prints the frame contents and hit/FAULT, followed by a 3/4/5-frame table (hit ratio printed with integer math, since xv6 has no float printf).

Frames	FIFO faults	FIFO hit ratio	LRU faults	LRU hit ratio
3	**15**	25.0%	**12**	40.0%
4	10	50.0%	8	60.0%
5	9	55.0%	7	65.0%

What it shows: the expected 15 (FIFO) and 12 (LRU) faults for 3 frames are reproduced. LRU is better at every size because the string has strong locality (0, 2, 3 recur), and LRU keeps recently used pages while FIFO evicts by arrival time even when a page is hot. For example, at the 6th reference (page 3) FIFO evicts page 0, which was used one step earlier, and faults on 0 at the very next reference, whereas LRU evicts page 1 and gets a hit. With user input `1,2,3,4,1,2,5,1,2,3,4,5`, FIFO gives 9 faults with 3 frames but **10 with 4 frames (Belady's anomaly)**, while LRU, a stack algorithm, drops from 10 to 8.